Security Plan: Pothole Detection System

Raspberry Pi Pipeline, Cloud Backend, and Frontend Applications

Capstone Project Team

July 2026

Abstract

This security plan identifies threats across the Raspberry Pi vision pipeline, the Vercel/Supabase cloud backend, and the Streamlit/React frontends; prioritizes risks by likelihood and impact; defines authentication, authorization, input validation, and data-security controls; and documents testing techniques, tools, findings, mitigations, accepted risks, and re-test closure criteria aligned to the course marking rubric.

Contents

1 Scope and System Boundary

This report covers three cooperating parts of the pothole detection system:

1. the Raspberry Pi vision-pipeline code presently in `rpi_model/` and the planned SIM/cellular upload function;
2. the cloud backend: a Vercel Next.js API (`api/`) backed by Supabase (Postgres table `pothole_reports` and Storage bucket `pothole-images`); and
3. the frontend applications that talk to that API: the Streamlit YOLO detection app (`backend/`) and the React map dashboard (`dashboard/`).

Sensitive assets in scope are camera frames, annotated frames, pothole detections, location data, model files, device and API credentials, Supabase service secrets, map imagery URLs, and service availability.

1.1 Data Flow

```
Camera -> Raspberry Pi processing -> local Flask dashboard
                                  -> [planned] SIM/cellular HTTPS upload
                                                -> Vercel API -> Supabase

Uploaded media -> Streamlit YOLO -> HTTPS POST (write key)
                                          -> Vercel API -> Supabase

React dashboard <- HTTPS GET (read key) <- Vercel API <- Supabase
```

Hardware enclosure and physical deployment remain a placeholder for the hardware engineer (Section 12).

2 Threat Identification and Risk Prioritization

2.1 Threat Identification Method

Threats were identified by reviewing attack surfaces on each trust boundary: local network exposure of the Pi Flask dashboard, outbound cellular upload, public HTTPS API routes, browser-origin calls to the API, uploaded image/ metadata payloads, and direct Supabase client access. Candidate threats include missing authentication, privilege abuse, injection, insecure transport, secret leakage, denial of service, and supply-chain compromise.

2.2 Risk Prioritization Method

Each threat is scored as Low, Medium, or High for **likelihood** and **impact**. **Priority** is derived as follows:

- **Critical**: High likelihood and High impact, or any path that yields unauthenticated write access to production data/storage;
- **High**: High impact with at least Medium likelihood, or High likelihood with Medium impact;
- **Medium**: Medium/Medium or lower combinations that still require tracked remediation or accepted-risk justification.

Every risk includes a **testing focus** so verification maps directly to the threat.

2.3 Pi and Cellular Upload Risk Register

Table 1: Pi and cellular-upload risk register

Risk / threat	Likelihood	Impact	Priority	Testing focus
Unauthenticated users can view the Pi dashboard, raw/annotated streams, and detection log.	High	High	Critical	Request every Flask route without credentials from a separate network client.
Camera frames and detection/location data travel over HTTP rather than HTTPS.	High	High	Critical	Capture traffic; confirm HTTP is refused in deployment.
Many concurrent MJPEG streams exhaust Pi CPU, memory, or bandwidth.	High	High	High	Concurrent-connection and bandwidth tests; verify connection limits.
Plaintext location lookup (`ip-api.com`) is untrusted and injectable into the UI.	Medium	High	High	Simulate modified, malformed, slow, and unavailable responses; XSS payload test.
Planned cellular uploader accepts forged, replayed, or modified requests.	Medium	High	High	Invalid credential, altered body, reused timestamp/nonce/request-ID tests.

Risk / threat	Likelihood	Impact	Priority	Testing focus
Lost or copied device credentials submit false detections.	Medium	High	High	Per-device revocation and replacement test.
Unbounded configuration or malformed local inputs crash or starve the pipeline.	Medium	Medium	Medium	Boundary tests for dimensions, JPEG quality, inference settings, paths.
Unpinned dependencies or untrusted model checkpoints enable supply-chain compromise.	Medium	High	High	Dependency audit and model checksum/provenance verification.
Saved output frames fill local storage when `SAVE_OUTPUT` is enabled.	Medium	Medium	Medium	Retention/quota and full-disk recovery tests.

2.4 Cloud Backend and Frontend Risk Register

Table 2: Cloud backend and frontend risk register

Risk / threat	Likelihood	Impact	Priority	Testing focus
Unauthenticated clients can `POST`/`PATCH`/`DELETE` reports and upload images.	High	High	Critical	Mutation routes without `X-API-Key`; expect 401.
Unauthenticated clients can enumerate reports via `GET /api/reports`.	High	Medium	High	GET without read key; expect 401 when `API_READ_KEY` is set.
Any website can call the API from a browser (CORS `*`).	High	High	High	OPTIONS/GET from an unlisted Origin; expect denial.
Upload endpoints accept spoofed `Content-Type` or oversized payloads.	Medium	High	High	Non-image bytes labelled JPEG; uploads >10 MB.

Risk / threat	Likelihood	Impact	Priority	Testing focus
Empty/fabricated detections become map markers and storage clutter.	Medium	Medium	High	POST with `detection_count=0`; expect 400.
Supabase table readable/writable with a leaked anon key if RLS is off.	Medium	High	High	Query table with anon key after RLS enablement.
Write key or Supabase secret appears in frontend bundles or git.	Medium	Critical	Critical	Secret scan; confirm dashboard ships only the read key.
API abuse exhausts bandwidth, storage, or function quotas.	High	High	High	Burst requests; verify HTTP 429.
Production 500 responses leak internal exception text.	Medium	Medium	Medium	Forced server error; expect generic message.
XSS via unsanitized dashboard rendering of API/location text.	Medium	High	High	Inject script payloads into notes/location fields; verify text-only render.
SQL injection via report filters, IDs, or metadata if raw SQL were used.	Low	High	Medium	Parameterized ORM/client checks; malicious ID/query strings.
Command injection via filenames or configuration reaching a shell.	Low	High	Medium	Special-character filenames and env values; confirm no shell execution.

3 Authentication and Authorization

3.1 Current Findings: Weak or Missing Authentication

Raspberry Pi Flask dashboard. The Pi application currently has **no authentication**, passwords, tokens, roles, or route-level authorization. It binds to `0.0.0.0`, exposing:

- `/` — dashboard;
- `/stream/original.mjpg` — raw camera feed;

- `/stream/edges.mjpg` — annotated camera feed;
- `/detections.json` — recent detections and location text.

This is a **Critical** finding if the Pi is reachable beyond a trusted maintenance network. Until authentication is added, access must be limited to localhost or a firewall/VPN-protected network.

Cloud API (implemented). The Vercel API previously accepted unauthenticated writes. It now enforces API-key authentication:

- **Write privilege**: `API_WRITE_KEY` required for `POST`/`PATCH`/`DELETE` and `/api/reports/backfill-lo`
- **Read privilege**: `API_READ_KEY` required for `GET /api/reports` and `GET /api/reports/[id]` when configured (enforced by the reviewed source code);
- **Health**: `GET /api/health` remains unauthenticated but rate-limited.

Keys are compared with constant-time equality. Clients send `X-API-Key` or `Authorization: Bearer`.

3.2 Access Privileges

Table 3: Access privilege matrix

Principal	Credential / channel	Privileges
Anonymous network client (Pi)	None	Full local dashboard/stream access today (unacceptable for public networks).
Streamlit detection app	`POTHOLE_API_WRITE_KEY`	Create reports only (POST multipart).
React dashboard / browser	`VITE_API_READ_KEY`	Read reports only (GET). No mutate.
Vercel API process	`SUPABASE_SECRET_KEY`	Server-side insert/update/delete/storage; never shipped to browsers.
Browser with Supabase anon key	Publishable/anon key	No table access when RLS is enabled with no public policies.
Future Pi cellular device	Per-device revocable credential	Upload only to fixed HTTPS ingestion URL; no inbound server role.

3.3 Password Entropy

- **Pi dashboard today:** password entropy is *not applicable* because no password login exists. If a local password is introduced, it must be hashed with Argon2id or bcrypt, rate-limited, never stored in source, and meet a minimum entropy policy (at least 12 characters with mixed classes, or a $\geq$128-bit random secret).
- **Cloud API keys:** not user passwords; they are long random secrets (`secrets.token_urlsafe`, $\geq$24–32 bytes). Entropy derives from cryptographic randomness rather than human-chosen passwords. Keys are rotated by regenerating and updating Vercel/local env vars.
- **Future end-user accounts:** if Supabase Auth/OAuth is added, enforce provider password policy or SSO, MFA for operators, and hashed storage managed by the auth provider.

3.4 Authentication and Authorization Test Cases

Table 4: Authentication and authorization tests

ID	Test	Expected result
A-01	Call every Pi Flask route with no credentials from another host.	Today: access succeeds (documented vulnerability). Target: denial after auth is added.
A-02	`GET /api/reports` with no key.	HTTP 401.
A-03	`POST /api/reports` with no key.	HTTP 401.
A-04	`GET /api/reports` with valid read key.	HTTP 200.
A-05	`POST /api/reports` with read key only.	HTTP 401 (read cannot elevate to write).
A-06	`DELETE /api/reports/[id]` with write key missing/wrong.	HTTP 401.
A-07	Dashboard bundle/source review for write/secret keys.	Only read key present; no Supabase secret.
A-08	Streamlit post with write key unset.	Client refuses before network call.
A-09	(Future) weak Pi password / brute-force attempts.	Lockout/rate-limit; hashes only stored.

Assessment status: No production endpoints were contacted. Source review identifies the expected 401/200 authentication behavior; it must be verified in an authorized local or staging environment.

4 Input Validation and Injection Testing

4.1 Validation Strategy: Client-Side and Server-Side

Security-critical validation is enforced on the **server** (Vercel API / future cloud ingestion). Clients may pre-check for usability, but are never trusted:

- **Streamlit (client):** only posts when detections contain "pothole"; refuses to post without a write key; trims notes.
- **React dashboard (client):** filters markers to records with coordinates and `detectionCount > 0`; does not accept free-form write inputs to the API.
- **API (server):** validates content type, image magic bytes, size ($\leq$10 MB), metadata JSON shape/size, severity/status enums, notes length ($\leq$2000), and rejects empty detections with HTTP 400.
- **Pi (local):** must add range checks for env-derived numeric config (dimensions, JPEG quality 1–100, confidence 0–1, etc.).

4.2 Injection Attack Surfaces

- **SQL injection.** The API uses the Supabase client with parameterized queries/filters (no string-concatenated SQL). Direct SQL from browsers is blocked by RLS. Regression tests still send malicious IDs and query strings.
- **Cross-site scripting (XSS).** Pi dashboard historically used `innerHTML` for external location text — high risk. Cloud notes/metadata must be rendered as text in the React UI. CORS allowlisting reduces hostile-site scripted API abuse from browsers.
- **Command injection.** Neither Pi nor API currently shells out on user input; filenames are sanitized. Tests still use `; rm -rf`, backticks, and newline names to confirm no shell path exists.
- **Other input abuse.** Oversized JSON, type confusion (array instead of object metadata), spoofed MIME types, path traversal in filenames.

4.3 Input Validation and Injection Test Cases

Table 5: Input validation and injection tests

ID	Test vector	Expected result
I-01	POST metadata with `detection_count=0`.	HTTP 400; no row created.

ID	Test vector	Expected result
I-02	Upload `.exe`/HTML bytes with `Content-Type: image/jpeg`.	Rejected (magic-byte mismatch).
I-03	Upload image >10 MB.	Rejected.
I-04	Metadata JSON array or oversized blob.	HTTP 400.
I-05	Invalid severity/status strings.	HTTP 400.
I-06	Notes with `<script>alert(1)</script>`.	Stored only if authorized write; dashboard renders as text, does not execute.
I-07	Pi location response containing script markup.	Must display via `textContent`; script must not run.
I-08	Report ID `1; DROP TABLE pothole_reports;-`.	No SQL execution; 404/validation error only.
I-09	Filename `../../etc/passwd;id.jpg`.	Sanitized object key; no path escape / no shell.
I-10	Extreme Pi env values (negative sizes, quality 999).	Safe rejection or clamped config (target after PI-07).

5 Data Security and Privacy Checks

5.1 Secure Transmission (HTTPS / TLS)

- **Cloud API and frontends:** the configured cloud endpoint uses HTTPS (`https://api-mu-ten-54.` Streamlit posts and dashboard fetches must target HTTPS URLs; no HTTP fallback for cloud calls.
- **Pi today:** local Flask dashboard and `http://ip-api.com` location lookup use plaintext HTTP — a Critical data-exposure risk on untrusted networks.
- **Planned SIM upload:** HTTPS only, with certificate validation; HTTP forbidden.

5.2 Encryption in Transit and at Rest

- **In transit:** TLS terminated by Vercel and Supabase for API and database/storage connections from the server.
- **At rest:** managed encryption at rest provided by Supabase / underlying cloud storage for Postgres data and bucket objects.
- **Secrets at rest on developer machines:** stored in gitignored `.env` / `.env.local` files and Vercel encrypted environment variables—not in the repository.

5.3 Data Leakage and Sensitive Information Exposure

Sensitive classes include camera imagery (people, plates, addresses), GPS / temporary location, API keys, and Supabase secrets.

Controls and checks:

- Frontends never embed `SUPABASE_SECRET_KEY` or `API_WRITE_KEY`.
- Source review indicates that API errors return generic messages (no stack traces or provider internals); verify this in staging.
- Rate limits and auth reduce anonymous bulk export of reports.
- Pi must not expose streams on public/cellular networks until auth and TLS exist; minimize retained frames and define retention before enabling `SAVE_OUTPUT`.
- Public storage URLs are an accepted demo risk with compensating write-path controls; private signed URLs are the longer-term fix.

5.4 Data Security Test Cases

Table 6: Data security and privacy tests

ID	Test	Expected result
D-01	Confirm production API URL scheme is HTTPS.	`https://` only.
D-02	Attempt HTTP cleartext to production API.	Redirect or connection failure; no successful cleartext API use.
D-03	Capture Streamlit upload and dashboard fetch.	TLS; no write key in dashboard traffic.
D-04	Force API 500 in production.	Generic error body; no secret/stack leakage.
D-05	Search git history / bundles for secret patterns (Gitleaks).	No live secrets committed.
D-06	Query Supabase with anon key after RLS.	Access denied to `pothole_reports`.
D-07	Pi traffic capture on LAN.	Today: plaintext imagery/location visible (open finding PI-02).

6 Testing Techniques and Tools

This section describes *how* the threats in Section 2 are verified. Techniques are applied across Pi, API, and frontends.

6.1 Static Application Security Testing (SAST)

SAST analyzes source without executing it. Planned/used tools:

- **Bandit** — Python SAST for the Pi and Streamlit code (unsafe functions, hard-coded secrets, weak crypto patterns).
- **Semgrep** — rule-based SAST for Python/TypeScript (Flask security, insecure HTTP, dangerous DOM sinks).
- **ESLint security plugins / TypeScript compiler** — dashboard and API static checks.
- **Ruff** — already used for Pi linting; complements SAST hygiene.

Evidence: clean reports or documented false positives / accepted risks.

6.2 Dynamic Application Security Testing (DAST)

DAST exercises a running service with crafted HTTP requests:

- **OWASP ZAP** — automated spider/active scan against the Pi Flask dashboard (when safely reachable on a lab network) and against a staging copy of the cloud API.
- **Burp Suite** — manual intercept/replay of API requests to tamper with headers, bodies, cookies/tokens, and CORS preflights.
- **curl / HTTPie scripts** — repeatable regression checks for 401/403/400/429 behaviour on staging and local API.

DAST focuses on missing authentication, privilege escalation (read key used as write), injection payloads, TLS downgrade attempts, and information leakage in responses.

6.3 Manual Code Review

Reviewers inspect Flask routes, Next.js API handlers (`api/lib/auth.ts`, `storage.ts`, `http.ts`), Streamlit upload client, React fetch paths, logging, model loading, and systemd/service configuration. Checklist items: secrets in source, auth on every mutating route, validation before persistence, safe rendering, and privilege separation.

6.4 Input Validation Checks

Covered in Section 4: boundary values, type confusion, magic-byte mismatches, and injection strings are sent to both client pre-checks (expect soft failure) and server endpoints (expect hard rejection).

6.5 Authentication and Authorization Testing

Covered in Section 3: missing credentials, wrong credentials, wrong privilege class, and future password-entropy/brute-force cases.

6.6 Additional Supporting Tools

Table 7: Security tools and required evidence

Tool / technique	Application	Required evidence
Manual code review	Pi, API, Streamlit, dashboard	Review notes linked to finding IDs.
Bandit / Semgrep (SAST)	Python/TS sources	Scan report or waived findings.
OWASP ZAP (DAST)	Pi dashboard; staging API	Scan summary + false-positive review.
Burp Suite (DAST)	API request tampering	Request/response exports for A-/I-/D-cases.
`pip-audit` / `npm audit`	Dependencies	Audit output; pinned versions.
Gitleaks	Repo + history	No unreviewed secrets.
curl regression scripts	Staging API auth/CORS/validation	Timestamped command output.
Load / concurrency tests	Pi MJPEG; API rate limits	Resource metrics; HTTP 429 proof.

7 Subsystem Detail: Raspberry Pi and Planned SIM Upload

7.1 Pi Input Validation and Injection (Current Code)

Numeric environment values are converted with `int()`/`float()` without range checks. Extreme values can exhaust resources. The location lookup response is inserted with JavaScript `innerHTML`, enabling XSS if the HTTP response is tampered with. There are presently no SQL or shell sinks on user input; those remain regression requirements if such features appear.

7.2 Pi Data Security and Availability

Plaintext HTTP for the dashboard and location service exposes imagery and location. Frames may contain PII; retention must be minimized. Uncapped MJPEG viewers create

denial-of-service risk. systemd hardening should use a non-root account, `NoNewPrivileges=true`, and narrowly writable output paths.

7.3 Planned SIM/Cellular Upload Requirements

The SIM module is outbound-only to a fixed HTTPS cloud URL. Each Pi needs a unique revocable credential (not a shared fleet key), TLS, timestamp + nonce / request ID, body integrity (e.g. signed hash), and rotation/revocation. The cloud must reject unknown devices, bad signatures, stale timestamps, duplicate nonces, and oversized/invalid payloads. Queuing uses bounded retries and must never log credentials.

8 Subsystem Detail: Backend and Cloud Data Service

Owner: Backend engineer (Amir Abdalla).

Configured cloud endpoint: `https://api-mu-ten-54.vercel.app`. Source: `api/`.

8.1 Implemented Controls

1. Least privilege: browsers never receive the Supabase secret.
2. Separate read/write API keys with fail-closed writes.
3. CORS Origin allowlist (local Vite defaults + `CORS_ALLOWED_ORIGINS`).
4. Per-IP rate limiting (stricter on writes); HTTP 429 + `Retry-After`.
5. Upload hardening: JPEG/PNG/WEBP, 10 MB max, magic-byte verification, sanitized UUID object keys.
6. Schema checks: metadata caps, enums, notes length, empty-detection rejection.
7. Generic API 500 responses (to be verified in staging).
8. RLS script for `pothole_reports` (`api/supabase/rls_pothole_reports.sql`).

8.2 Planned Backend Verification

- Unauthenticated GET/POST in staging → HTTP 401.
- Authorized GET with read key → HTTP 200.
- Write key + empty detections → HTTP 400.
- CORS preflight from unlisted Origin → HTTP 403.
- CORS preflight from `http://localhost:5173` → HTTP 204 with allowlisted ACAO header.

9 Subsystem Detail: Frontend Integration

Owner: Frontend / detection-app engineers.

- Streamlit loads `POTHOLE_API_WRITE_KEY` from `backend/.env` (never shown in UI) and posts over HTTPS.
- Dashboard uses `VITE_API_READ_KEY` only; GET-only; no Supabase admin client.
- Client-side filters complement, but do not replace, server validation.
- Accepted limitation: the Vite read key is visible in the browser bundle and is treated as a shared team token, not a root secret (see AR-01).

10 Reporting and Risk Mitigation Plan

10.1 Vulnerability and Finding Register

Findings are tracked with severity, status, owner, and closure evidence. Status values: **Open**, **Mitigated**, **Accepted** (with justification), or **Closed** (fix + re-test passed).

Table 8: Pi/SIM findings and mitigations

ID	Severity	Status	Required mitigation	Closure / re-test evidence
PI-01	Critical	Open	Restrict dashboard network; add authentication/authorization.	Unauthenticated route tests denied; proxy/firewall reviewed.
PI-02	Critical	Open	HTTPS for all remote connections; replace plaintext location lookup.	Traffic capture shows TLS only.
PI-03	High	Open	Render location with DOM text APIs; validate response schema.	XSS payload displays as text only.
PI-04	High	Open	Cap MJPEG viewers; monitor resources.	Load test within agreed limits.
PI-05	High	Open	Per-device SIM auth, integrity, replay protection, revocation.	Forged/stale/duplicate uploads rejected.
PI-06	High	Open	Pin/audit dependencies; verify model checksums.	Manifest + audit + checksum log.
PI-07	Medium	Open	Validate configuration ranges; safe failure messages.	Automated boundary tests pass.

ID	Severity	Status	Required mitigation	Closure / re-test evidence
PI-08	Medium	Open	Output retention/quota; least-privilege service account.	Full-disk/retention tests; permission review.

Table 9: Cloud/frontend findings and mitigations

ID	Severity	Status	Required mitigation	Closure / re-test evidence
BE-01	Critical	Mitigated	Write API key on mutations; fail closed.	Staging POST without key → 401.
BE-02	High	Mitigated	Read API key on report GETs.	Staging GET without key → 401; with key → 200.
BE-03	High	Mitigated	CORS allowlist instead of *.	Unlisted staging Origin → 403; local Vite allowed.
BE-04	High	Mitigated	Per-IP rate limits on API routes.	Staging burst test returns 429 (in-memory limiter).
BE-05	High	Mitigated	Magic-byte + type + size upload checks.	Spoofed/oversized uploads rejected in staging.
BE-06	High	Mitigated	Enable Supabase RLS; secret key server-only.	SQL script provided; confirm applied in project.
BE-07	Medium	Mitigated	Cap metadata/notes; reject empty detections.	Empty detection POST → 400 in staging.
BE-08	Medium	Mitigated	Generic API 500 bodies.	No stack/provider leakage in staging errors.
FE-01	High	Mitigated	Streamlit write key from env only.	Staging test: missing key blocks post; secret not in UI.
FE-02	High	Mitigated	Dashboard read-only API use.	Source review + staging GET-only traffic.
FE-03	Medium	Accepted	Document bundlable read key as team token.	See AR-01.

10.2 Accepted Risks (with Justification)

Table 10: Accepted risks

ID	Risk	Justification / compensating control	Review date
AR-01	Dashboard read key visible in JS bundle.	Mid-project shared demo; key is read-only; writes still fail closed; rotate if leaked.	End of Week 12
AR-02	Public Supabase image URLs.	Writes API-gated; RLS on table; migrate to signed URLs before public launch.	Before public demo
AR-03	No end-user login / RBAC yet.	Operator-only demo; API keys separate read/write; track Auth as future work.	Final delivery
AR-04	In-memory API rate limiter.	Adequate for single-region demo; distributed limiter/WAF later.	Production hardening

10.3 Re-testing and Issue Closure Process

An issue is **Closed** only when all of the following are true:

1. The fix is implemented and code-reviewed.
2. The mapped test IDs (A-/I-/D- or table testing focus) are re-run.
3. Evidence (request/response, scan excerpt, or review note) is attached to the finding ID.
4. No regression appears in subsequent CI/manual smoke tests of auth, validation, and TLS.

Accepted risks require an owner, justification, compensating control, and a scheduled review date (Table 10). Open Pi items (PI-01–PI-08) remain the active remediation backlog for the edge device and SIM path.

11 Deferred Production Hardening

- End-user authentication (Supabase Auth / OAuth) and RBAC views.
- Per-device Pi credentials with nonce/replay protection on cellular upload.

- Private storage with short-lived signed URLs.
- Distributed rate limiting / WAF; CI-wired Bandit, Semgrep, ZAP, and Gitleaks.

12 Hardware and Physical Deployment

Hardware and physical-deployment testing was outside the read-only Phase 1 assessment: no device, camera, SIM, or deployed network was accessed. Before deployment, the team must document and review physical tamper controls, power protection, camera/privacy placement, cellular SIM management, exposed-port controls, lost-device response, and maintenance procedures.

A Evidence Package

Assessment evidence is stored in `security-evidence/`.

Table 11: Security assessment evidence package

Evidence area	Contents	Files
Assessment overview	Scope, handling, status, and summary	`README.md`; `executive-summary.md`
Scope and review	System, data-flow, route, and manual-review records	`00-scope/`; `01-manual-review/`
Scan and dependency review	SAST, dependency, and secret-review outputs	`02-sast/`; `03-dependencies/`; `04-secrets/`
Planned test procedures	Authentication through availability test plans	`05-authentication/` through `09-availability/`
Results and closure	Test results, findings, remediation, and re-test records	`test-results.csv`; `findings-register.csv`; `10-findings/`; `11-closure/`